

PPS - Scala Stream Fusion

Leonardo Naddei

26 luglio 2026

Indice

1	Requirement specification	4
1.1	Requisiti di business	4
1.2	Modello di dominio	4
1.3	Requisiti funzionali utente	4
1.4	Requisiti funzionali di sistema	4
1.5	Requisiti non funzionali	4
1.6	Requisiti di implementazione	4
2	Design architetturale	4
3	Design di dettaglio	4
4	Implementazione	4
5	Testing	4
6	Retrospettiva	4

Abstract

Il presente lavoro descrive la realizzazione di una libreria per Scala 3 che consenta di definire pipeline di elaborazione secondo un paradigma dichiarativo, ottenendo al contempo prestazioni comparabili a quelle di codice imperativo specializzato manualmente.

Basandosi sul concetto di stream fusion, introdotto in Stream Fusion: From Lists to Streams to Nothing at All, e sfruttando le funzionalità di metaprogrammazione offerte dalle macro di Scala 3, è stata sviluppata una libreria capace di trasformare pipeline dichiarative in codice specializzato ed efficiente generato a compile-time. La libreria offre inoltre un'architettura estendibile per la definizione di nuovi operatori terminali, permettendo di integrare nuovi comportamenti nella fase di fusione senza rinunciare alle ottimizzazioni statiche.

Tale generazione statica permette di eliminare i principali overhead tipici delle astrazioni funzionali su JVM. L'analisi del bytecode prodotto conferma la completa fusione delle operazioni sequenziali in un unico ciclo imperativo e l'inlining diretto delle lambda nel callsite. Ciò azzerava l'uso di istruzioni `invokedynamic`, elimina il dispatch indiretto (evitando call site polimorfici o megamorfici) e rimuove le allocazioni di collezioni intermedie, iteratori e closure, azzerando al contempo il boxing dei tipi primitivi.

Per certi aspetti, il modello ottenuto richiama il paradigma di genericità dei template C++: il codice viene specializzato in fase di compilazione rispetto ai tipi e alla specifica composizione delle operazioni, quando tali informazioni sono disponibili staticamente. Le macro di Scala aggiungono tuttavia una componente esplicita di metaprogrammazione, permettendo non soltanto di istanziare codice generico, ma anche di analizzare, trasformare e ottimizzare la struttura della pipeline prima della sua emissione.

I benchmark condotti tramite JMH mostrano uno speedup significativo e un consumo di memoria drasticamente ridotto rispetto alle librerie standard. Il risultato è un framework che preserva l'espressività, la composizionalità e la type safety tipiche della programmazione funzionale.

1 Requirement specification

1.1 Requisiti di business

1.2 Modello di dominio

1.3 Requisiti funzionali utente

1.4 Requisiti funzionali di sistema

1.5 Requisiti non funzionali

1.6 Requisiti di implementazione

2 Design architetturale

L'applicativo è composto da due aree: l'API utilizzabile e un compilatore che si occupa del parsing, ottimizzazione e creazione di codice ad hoc a partire dal codice delle macro

3 Design di dettaglio

design dell'API

- alberi di parsing

- estensione per enrichment

4 Implementazione

algoritmo implementato (push stream)

- ottimizzazioni applicate ai parametri

- enrichment: bubbling delle dichiarazioni

- generazione di codice

- implementazione del filter

- implementazione del flatmapping

- es semplice:

- `IncIter(from: Int): infinite lazy iterator, each next returns the next`

- `FusedStream .from(List(1,2,3,4,5)) .skip(3) .flatMap(n -> Stream.of(IncIter(n*10)).limit(3))`

- `.limit(10) .toList()`

- `// Must be converted to`

- `i = 0 j = 0`

- `for i in 1: if(i++ >= 3) «< must inject condition on j too k = 0 iter = IncIter( i* 10) for`

- `ii in iter if(k++ >= 3) break ——— This get injectd into the flatMap ——— if(j ++ >= 10)`

- `break all push(ii) ———`

- esempio estensione con collector utilizzando i trait

5 Testing

6 Retrospettiva